

■ Terminologia

- adiamento indefinido 233
- aritmética de ponteiro 217
- array de ponteiros 232
- array de strings 232
- chamada por referência 212
- chamada por valor 212
- deslocamento de um ponteiro 228
- desreferenciação de um ponteiro 211
- dilema de tempo/espaço 219
- estrutura 219
- indireção 209
- notação de ponteiro/deslocamento 228
- notação de ponteiro/subscrito 228
- operador de endereço (&) 210
- operador de desreferenciação (*) 211
- operador de indireção (*) 211
- ponteiro 209
- ponteiro constante para dados constantes 216
- ponteiro constante para dados não constantes 216
- ponteiro não constante para dados constantes 216
- ponteiro não constante para dados não constantes 216
- ponteiro para função 239
- ponteiro para uma função 237
- ponteiro para void (void *) 227
- princípio do menor privilégio 216
- qualificador const 215
- registro 219
- simulação de chamada por referência 211
- sizeof 223
- subscrito de ponteiro 229
- tipo `size_t` 224
- void * (ponteiro para void) 227

■ Exercícios de autorrevisão

- 7.1** Preencha os espaços em cada uma das sentenças:
- a)** Uma variável de ponteiro contém como valor o(a) _____ de outra variável.
 - b)** Os três valores que podem ser usados para inicializar um ponteiro são _____, _____ e _____.
 - c)** O único inteiro que pode ser atribuído a um ponteiro é _____.
- 7.2** Indique se as seguintes sentenças são *falsas* ou *verdadeiras*. Se a resposta for *falsa*, explique.
- a)** O operador de endereço (&) pode ser aplicado somente a constantes, a expressões e a variáveis declaradas com a classe de armazenamento `register`.
 - b)** Um ponteiro que é declarado como `void` pode ser desreferenciado.
 - c)** Ponteiros de diferentes tipos podem ser atribuídos uns aos outros sem uma operação de coerção (`cast`).
- 7.3** Resolva cada um dos itens a seguir. Suponha que os números em ponto flutuante com precisão simples sejam armazenados em 4 bytes, e que o endereço inicial do array esteja no local 1002500 na memória. Cada parte do exercício deverá usar os resultados das partes anteriores, se isso for apropriado.
- a)** Defina um array do tipo `float` chamado `numbers` com 10 elementos, e inicialize os elementos com os valores 0.0, 1.1, 2.2, ..., 9.9. Suponha que a constante simbólica `SIZE` tenha sido definida como 10.
 - b)** Defina um ponteiro, `nPtr`, que aponte um objeto do tipo `float`.
 - c)** Imprima os elementos do array `numbers` utilizando a notação de subscrito de array. Use uma instrução `for` e suponha que a variável de controle inteira `i` tenha sido definida. Imprima cada número com 1 posição de precisão à direita do ponto decimal.
 - d)** Indique duas instruções separadas que atribuam o endereço inicial do array `numbers` à variável de ponteiro `nPtr`.
 - e)** Imprima os elementos do array `numbers` usando a notação ponteiro/deslocamento com o ponteiro `nPtr`.
 - f)** Imprima os elementos do array `numbers` usando a notação de ponteiro/deslocamento com o nome de array como ponteiro.
 - g)** Imprima os elementos do array `numbers` subscrutando o ponteiro `nPtr`.
 - h)** Faça referência ao elemento 4 do array `numbers` usando a notação de subscrito de array, a notação de ponteiro/deslocamento com o nome do array como ponteiro, a notação de subscrito de ponteiro com `nPtr` e a notação de ponteiro/deslocamento com `nPtr`.
 - i)** Supondo que `nPtr` aponte para o início do array `numbers`, qual endereço é referenciado por `nPtr + 8`? Que valor é armazenado nesse local?

- j) Supondo que `nPtr` aponte para `numbers[5]`, qual endereço é referenciado por `nPtr -= 4`? Qual o valor armazenado nesse local?

7.4 Escreva instruções que realizem as tarefas requeridas para cada um dos itens a seguir. Suponha que as variáveis de ponto flutuante `number1` e `number2` sejam definidas, e que `number1` seja inicializado em 7.3.

- Defina a variável `fPtr` como um ponteiro para um objeto do tipo `float`.
- Atribua o endereço da variável `number1` à variável de ponteiro `fPtr`.
- Imprima o valor do objeto apontado por `fPtr`.
- Atribua o valor do objeto apontado por `fPtr` à variável `number2`.
- Imprima o valor de `number2`.
- Imprima o endereço de `number1`. Use o especificador de conversão `%p`.
- Imprima o endereço armazenado em `fPtr`. Use o especificador de conversão `%p`. O valor impresso é o mesmo que o endereço de `number1`?

7.5 Execute as seguintes tarefas:

- Escreva um protótipo de função para uma função chamada `exchange`, que usa dois ponteiros para números em ponto flutuante `x` e `y` como parâmetros e não retorna um valor.
- Escreva um protótipo de função para a função na parte (a).
- Escreva o protótipo da função para uma função chamada `evaluate` que retorna um inteiro e que usa

como parâmetros o inteiro `x` e um ponteiro para a função `poly`. A função `poly` usa um parâmetro inteiro e retorna um inteiro.

- d) Escreva o protótipo de função para a função na parte (c).

7.6 Encontre o erro em cada um dos segmentos de programa a seguir. Considere

```
int *zPtr; /* zPtr referenciará o array z */
int *aPtr = NULL;
void *sPtr = NULL;
int number, i;
int z[ 5 ] = { 1, 2, 3, 4, 5 };
sPtr = z;

a) ++zptr;
b) /* usa o ponteiro para obter o
   primeiro valor do array; suponha que
   zPtr seja inicializado */number = zPtr;
c) /* atribui o elemento do array 2 (o
   valor 3) a number;suponha que zPtr
   seja inicializado */number = *zPtr
   [ 2 ];
d) /* imprime array z inteiro; suponha
   que zPtr seja inicializado */
   for ( i = 0; i <= 5; i++ ) {
       printf( "%d ", zPtr[ i ] );}
e) /* atribui o valor apontado por sPtr a
   number */number = *sPtr;
f) ++z;
```

Respostas dos exercícios de autorrevisão

7.1 a) endereço. b) 0, NULL, um endereço. c) 0.

7.2 a) Falso. O operador de endereço pode ser aplicado somente a variáveis. O operador de endereço não pode ser aplicado a variáveis declaradas com classe de armazenamento `register`.

b) Falso. Um ponteiro para `void` não pode ser desreferenciado, pois não existe um modo de saber exatamente quantos bytes de memória desreferenciar.

c) Falso. Os ponteiros do tipo `void` podem receber ponteiros de outros tipos, e ponteiros do tipo `void` podem ser atribuídos a ponteiros de outros tipos.

7.3 a) `float numbers[ SIZE ] =`
`{ 0.0, 1.1, 2.2, 3.3, 4.4, 5.5, 6.6,`
`7.7, 8.8, 9.9 };`

b) `float *nPtr;`

c) `for ( i = 0; i < SIZE; i++ ) {`
`printf( "%.1f ", numbers[ i ] );`
`}`

d) `nPtr = numbers;`
`nPtr = &numbers[ 0 ];`

e) `for ( i = 0; i < SIZE; i++ ) {`
`printf( "%.1f ", *( nPtr + i ) );`
`}`

f) `for ( i = 0; i < SIZE; i++ ) {`
`printf( "%.1f ", *( numbers + i ) );`
`}`

g) `for ( i = 0; i < SIZE; i++ ) {`
`printf( "%.1f ", nPtr[ i ] );`
`}`

- h) `numbers[ 4 ]`
`*( numbers + 4 )`
`nPtr[ 4 ]`
`*( nPtr + 4 )`
- i) O endereço é $1002500 + 8 * 4 = 1002532$. O valor é 8.8.
- j) O endereço de `numbers[ 5 ]` é $1002500 + 5 * 4 = 1002520$.
 O endereço de `nPtr -= 4` is $1002520 - 4 * 4 = 1002504$.
 O valor nesse local é 1.1.

- 7.4**
- a) `float *fPtr;`
 b) `fPtr = &número1;`
 c) `printf( "O valor de *fPtr é %f\n", *fPtr );`
 d) `número2 = *fPtr;`
 e) `printf( "O valor do número2 é is %f\n", número2 );`
 f) `printf( "O endereço do número1 é %p\n", &número1 );`
 g) `printf( "O endereço armazenado em fptr é %p\n", fPtr );`
 Sim, o valor é o mesmo.

- 7.5**
- a) `void exchange( float *x, float *y )`
 b) `void exchange( float *x, float *y );`

- c) `int evaluate( int x, int (*poly)( int ) )`
 d) `int evaluate( int x, int (*poly)( int ) );`

7.6

- a) Erro: `zPtr` não foi inicializado.
 Correção: Inicialize `zPtr` com `zPtr = z;` antes de realizar a aritmética de ponteiro.
- b) Erro: O ponteiro não está desreferenciado.
 Correção: Mude a instrução para `number = *zPtr;`
- c) Erro: `zPtr[ 2 ]` não é um ponteiro e não deve ser desreferenciado.
 Correção: Mude `*zPtr[ 2 ]` para `zPtr[ 2 ]`.
- d) Erro: Referir-se a um elemento do array fora dos limites do array com subscrito de ponteiro.
 Correção: Mude o operador `<=` na condição de `for` para `<`.
- e) Erro: Desreferenciar um ponteiro void.
 Correção: Para desreferenciar o ponteiro, ele primeiro precisa ser convertido para um ponteiro inteiro.
 Mude a instrução para `number = *( ( int * ) sPtr );`
- f) Erro: Tentar modificar um nome de array com aritmética de ponteiro.
 Correção: Use uma variável de ponteiro em vez do nome do array para realizar a aritmética de ponteiro, ou crie um subscrito do nome do array para se referir a um elemento específico.

Exercícios

- 7.7** Preencha os espaços em cada uma das sentenças:
- a) O operador _____ retorna o local na memória em que seu operando está armazenado.
- b) O operador _____ retorna o valor do objeto ao qual seu operando aponta.
- c) Para simular a chamada por referência ao passar uma variável de não array a uma função, é necessário passar o(a) _____ da variável à função.
- 7.8** Indique se as seguintes sentenças são *falsas* ou *verdadeiras*. Se a resposta for *falsa*, explique.
- a) Dois ponteiros que apontam para diferentes arrays não podem ser comparados de modo significativo.
- b) Como o nome de um array é um ponteiro para o primeiro elemento do array, os nomes de array podem ser manipulados exatamente da mesma maneira que os ponteiros.
- 7.9** Execute as tarefas requeridas para cada um dos itens a seguir. Suponha que inteiros sem sinal sejam armazena-

dos em 2 bytes, e que o endereço inicial do array esteja no local 1002500 da memória.

- a) Defina um array do tipo `unsigned int` chamado `values` com cinco elementos, e inicialize os elementos para os inteiros pares de 2 a 10. Suponha que a constante simbólica `SIZE` tenha sido definida como 5.
- b) Defina um ponteiro `vPtr` que aponte para um objeto do tipo `unsigned int`.
- c) Imprima os elementos do array `values` usando a notação de subscrito de array. Use uma estrutura `for` e suponha que a variável de controle inteira `i` tenha sido definida.
- d) Mostre duas instruções separadas que atribuam o endereço inicial do array `values` para a variável de ponteiro `vPtr`.
- e) Imprima os elementos do array `values` usando a notação de ponteiro/deslocamento.

- f) Imprima os elementos do array `values` usando a notação de ponteiro/deslocamento com o nome de array como ponteiro.
- g) Imprima os elementos do array `values` subscritando o ponteiro para o array.
- h) Consulte o elemento 5 do array `values` usando a notação de subscrito de array, a notação de ponteiro/deslocamento com o nome do array como ponteiro, a notação de ponteiro/subscrito e a notação de ponteiro/deslocamento.
- i) Que endereço é referenciado por `vPtr + 3`? Que valor é armazenado nesse local?
- j) Supondo que `vPtr` aponte para `values[4]`, que endereço é referenciado por `vPtr -= 4`? Que valor é armazenado nesse local?

7.10 Escreva uma única instrução que execute as tarefas requeridas para cada um dos itens a seguir. Suponha que as variáveis inteiras longas `value1` e `value2` tenham sido definidas, e que `value1` tenha sido inicializado com 200000.

- a) Defina a variável `lPtr` para que ela seja um ponteiro para um objeto do tipo `long`.
- b) Atribua o endereço da variável `value1` à variável de ponteiro `lPtr`.
- c) Imprima o valor do objeto apontado por `lPtr`.
- d) Atribua o valor do objeto apontado por `lPtr` à variável `value2`.
- e) Imprima o valor de `value2`.
- f) Imprima o endereço de `value1`.
- g) Imprima o endereço armazenado em `lPtr`. O valor impresso é o mesmo que o endereço de `value1`?

7.11 Execute as tarefas a seguir:

- a) Escreva o cabeçalho de função para a função `zero`, que usa um parâmetro de array de inteiros longos `bigIntegers` e não retorna um valor.
- b) Escreva o protótipo de função para a função na parte a.
- c) Escreva o cabeçalho de função para a função `add1AndSum`, que usa um parâmetro de array de inteiros `oneTooSmall` e retorna um inteiro.
- d) Escreva o protótipo de função para a função descrita na parte c.

Nota: os exercícios 7.12 a 7.15 são consideravelmente desafiadores. Quando tiver resolvido esses problemas, deverá ser capaz de implementar com facilidade os jogos de cartas mais populares.

7.12 **Embaralhamento e distribuição de cartas.** Modifique o programa na Figura 7.24 de modo que a função de distribuição de cartas cuide de uma mão de cinco

cartas no pôquer. Depois, escreva as seguintes funções adicionais:

- a) Determine se a mão contém um par.
- b) Determine se a mão contém dois pares.
- c) Determine se a mão contém uma trinca (por exemplo, três valetes).
- d) Determine se a mão contém uma quadra (por exemplo, quatro ases).
- e) Determine se a mão contém um *flush* (ou seja, cinco cartas do mesmo naipe).
- f) Determine se a mão contém *straight* (ou seja, cinco cartas de valores de naipe consecutivos).

7.13 **Projeto: embaralhar e distribuir cartas.** Use as funções desenvolvidas no Exercício 7.12 para escrever um programa que trate duas mãos de pôquer com cinco cartas; avalie cada mão e determine qual é a melhor.

7.14 **Projeto: embaralhar e distribuir cartas.** Modifique o programa desenvolvido no Exercício 7.13 de modo que seja possível simular o papel do carteador. A mão de cinco cartas do carteador deve ser ‘virada para baixo’, de modo que o jogador não possa vê-la. O programa deve, então, avaliar a mão do carteador, e baseado na qualidade da mão, o carteador deve retirar mais uma, duas ou três cartas para substituir o número correspondente de cartas desnecessárias na mão original. O programa deve, então, reavaliar a mão do carteador. [Cuidado: este é um problema difícil!]

7.15 **Projeto: embaralhar e distribuir cartas.** Modifique o programa desenvolvido no Exercício 7.14 de modo que seja possível lidar com a mão do carteador automaticamente, mas deixando para o jogador a decisão de quais cartas ele deseja substituir. O programa deverá, então, avaliar as duas mãos e determinar quem vence. Agora, use esse novo programa para jogar 20 vezes contra o computador. Quem ganha mais jogos, você ou o computador? Peça a um de seus amigos que jogue 20 vezes contra o computador. Quem vence mais jogos? Com base nos resultados desses jogos, faça modificações apropriadas para refinar seu programa de jogo de pôquer (este também é um problema difícil). Jogue mais 20 vezes. Seu programa modificado joga melhor?

7.16 **Modificação no embaralhamento e na distribuição de cartas.** No programa de embaralhar e distribuir cartas da Figura 7.24, usamos intencionalmente um algoritmo de embaralhamento ineficaz, que oferecia a possibilidade de adiamento indefinido. Nesse problema, você criará um algoritmo de embaralhamento de alto desempenho, que evita o adiamento indefinido.

Modifique o programa da Figura 7.24 da seguinte forma. Comece inicializando o array `deck` como mostra a Figura 7.29. Modifique a função `shuffle` para que ela percorra linha por linha e coluna por coluna pelo array, tocando cada elemento uma vez. Cada elemento deve ser trocado por um elemento do array selecionado aleatoriamente. Imprima o array resultante para determinar se as cartas estão sendo embaralhadas satisfatoriamente (como na Figura 7.30, por exemplo). Você pode querer que seu programa chame a função `shuffle` várias vezes, garantindo assim um embaralhamento satisfatório.

Embora a técnica, nesse problema, aprimore o algoritmo de embaralhamento, o algoritmo de distribuição ainda requer a busca no array `deck` pela carta 1, depois pela carta 2, depois pela carta 3 e assim por diante. Pior ainda, mesmo depois de o algoritmo de distribuição localizar e distribuir a carta, o algoritmo continua pesquisando o restante do baralho. Modifique o programa da Figura 7.24 de modo que, quando uma carta for distribuída, nenhuma outra tentativa de combinar essa carta seja feita, e que o programa prossiga imediatamente com a

distribuição da próxima carta. No Capítulo 10, desenvolveremos um algoritmo de distribuição que exige apenas uma operação por carta.

7.17 Simulação: a tartaruga e a lebre. Nesse problema, você recriará um dos momentos realmente importantes da história, a saber, a corrida clássica entre a tartaruga e a lebre. Você usará a geração de números aleatórios para desenvolver uma simulação desse evento memorável.

Nossos competidores começam a corrida no ‘quadrado 1’ de 70 quadrados. Cada quadrado representa uma posição possível ao longo do curso a ser percorrido. A linha de chegada fica no quadrado 70. O primeiro concorrente a alcançar ou passar do quadrado 70 é recompensado com um balde de cenouras e alface. O trajeto é percorrido por uma montanha escorregadia, de modo que, ocasionalmente, os concorrentes perdem o contato com o chão.

Há um relógio que apresenta o tempo em os segundos. A cada segundo indicado pelo relógio, seu programa deverá ajustar a posição dos animais de acordo com as regras da Figura 7.31.

Array deck não embaralhado													
	0	1	2	3	4	5	6	7	8	9	10	11	12
0	1	2	3	4	5	6	7	8	9	10	11	12	13
1	14	15	16	17	18	19	20	21	22	23	24	25	26
2	27	28	29	30	31	32	33	34	35	36	37	38	39
3	40	41	42	43	44	45	46	47	48	49	50	51	52

Figura 7.29 ■ Array deck não embaralhado.

Exemplo do array deck embaralhado													
	0	1	2	3	4	5	6	7	8	9	10	11	12
0	19	40	27	25	36	46	10	34	35	41	18	2	44
1	13	28	14	16	21	30	8	11	31	17	24	7	1
2	12	33	15	42	43	23	45	3	29	32	4	47	26
3	50	38	52	39	48	51	9	5	37	49	22	6	20

Figura 7.30 ■ Exemplo de array deck embaralhado.

Animal	Tipo de movimento	Porcentagem do tempo	Movimento real
Tartaruga	Caminha rapidamente	50%	3 quadrados à direita
	Escorrega	20%	6 quadrados à esquerda
	Caminha lentamente	30%	1 quadrado à direita
Lebre	Dorme	20%	Não faz nenhum movimento
	Dá um salto grande	20%	9 quadrados à direita
	Escorrega bastante	10%	12 quadrados à esquerda
	Dá um salto pequeno	30%	1 quadrado à direita
	Escorrega pouco	20%	2 quadrados à esquerda

Figura 7.31 ■ Regras referentes aos movimentos feitos pela tartaruga e pela lebre usados no ajuste de posições.

Use variáveis para acompanhar as posições dos animais (ou seja, os números de posição vão de 1 a 70). O jogo deve começar com os dois animais na posição 1 (ou seja, 'na linha de partida'). Se um animal escorregar antes do quadrado 1, mova-o de volta para o quadrado 1.

Gere as porcentagens na tabela anterior produzindo um inteiro aleatório, i , na faixa de $1 \leq i \leq 10$. No caso da tartaruga, execute um 'caminha rapidamente' quando $1 \leq i \leq 5$, um 'escorrega' quando $6 \leq i \leq 7$, ou um 'caminha lentamente' quando $8 \leq i \leq 10$. Use uma técnica semelhante para mover a lebre.

Comece a corrida imprimindo

BANG !!!!!

E LÁ VÃO ELES !!!!!

Depois, para cada segundo marcado pelo relógio (ou seja, cada repetição de um loop), imprima uma linha de 70 posições mostrando a letra T na posição da tartaruga e a letra L na posição da lebre. Ocasionalmente, os competidores acabarão no mesmo quadrado. Nesse caso, a tartaruga morderá a lebre e seu programa deverá imprimir AI!!! a partir dessa posição. Todas as posições de impressão diferentes de T, de L, ou do AI!!! (no caso de um empate) devem estar em branco.

Depois que cada linha for impressa, verifique se um dos animais alcançou ou passou o quadrado 70. Nesse caso, imprima o vencedor e termine a simulação. Se a tartaruga vencer, imprima TARTARUGA VENCE!!! É ISSO AÍ!!! Se a lebre vencer, imprima Lembre vence. Marmelada. Se os dois animais vencerem na mesma batida do relógio, você pode favorecer a tartaruga (a 'pobre coitada') ou pode querer imprimir É um empate. Se nenhum animal vencer, realize o loop novamente para simular a próxima batida do relógio. Quando estiver pronto para executar seu programa, monte um grupo de fãs para assistir a corrida. Você ficará surpreso ao ver como eles se mostrarão interessados!

7.18 Modificação no embaralhamento e na distribuição de cartas. Modifique o programa de embaralhamento e distribuição de cartas da Figura 7.24 de modo que essas operações sejam realizadas pela mesma função (`shuffle` e `deal`). A função deverá conter uma estrutura de looping aninhada que é semelhante à função `shuffle` da Figura 7.24.

7.19 O que o programa a seguir faz?

```
1 /* ex07_19.c */
2 /* 0 que esse programa faz? */
3 #include <stdio.h>
4
5 void mystery1( char *s1, const char
6               *s2 ); /* protótipo */
```

```
6
7 int main( void )
8 {
9     char string1[ 80 ]; /* cria char
10    array */
11    char string2[ 80 ]; /* cria char
12    array */
13
14    printf( "Digite duas strings: " );
15    scanf( "%s%s", string1, string2 );
16    mystery1( string1, string2 );
17    printf( "%s", string1 );
18    return 0; /* indica conclusão bem-
19    sucedida */
20 } /* fim do main */
21
22 /* 0 que essa função faz? */
23 void mystery1( char *s1, const char
24               *s2 )
25 {
26     while ( *s1 != '\0' ) {
27         s1++;
28     } /* fim do while */
29
30     for ( ; *s1 = *s2; s1++, s2++ ) {
31         ; /* sem comando */
32     } /* fim do for */
33 } /* fim da função mystery1 */
```

7.20 O que o programa a seguir faz?

```
1 /* ex07_20.c */
2 /* 0 que esse programa faz? */
3 #include <stdio.h>
4
5 int mystery2( const char *s ); /*
6 protótipo */
7
8 int main( void )
9 {
10     char string[ 80 ]; /* cria array
11     char */
12
13     printf( "Digite uma string: " );
14     scanf( "%s", string );
15     printf( "%d\n", mystery2( string
16     ) );
17     return 0; /* indica conclusão bem-
18     sucedida */
19 } /* fim do main */
20
21 /* 0 que essa função faz? */
22 int mystery2( const char *s )
23 {
24     int x; /* contador */
25
26     /* loop pela string */
27     for ( x = 0; *s != '\0'; s++ ) {
```



```

24     x++;
25 } /* fim do for */
26
27 return x;
28 } /* fim da função mystery2 */

```

7.21 Encontre o erro em cada um dos segmentos de programa a seguir. Se o erro puder ser corrigido, explique como fazê-lo.

- a) `int *number;`
`printf( "%d\n", *number );`
- b) `float *realPtr;`
`long *integerPtr;`
`integerPtr = realPtr;`
- c) `int * x, y;`
`x = y;`
- d) `char s[] = "este é um array de caracteres";`
`int count;`
`for ( ; *s != '\0'; s++)`
`printf( "%c ", *s );`
- e) `short *numPtr, result;`
`void *genericPtr = numPtr;`
`result = *genericPtr + 7;`
- f) `float x = 19.34;`
`float xPtr = &x;`
`printf( "%f\n", xPtr );`
- g) `char *s;`
`printf( "%s\n", s );`

7.22 Travessia de labirinto. A grade a seguir é uma representação do array de duplo subscrito de um labirinto.

```

# # # # # # # # # # #
# . . . # . . . . . #
. . # . # . # # # . #
# # # . # . . . . # . #
# . . . . # # # . # . .
# # # # . # . # . # . #
# . . # . # . # . # . #
# # . # . # . # . # . #
# . . . . . . . # . #
# # # # # # # . # # . #
# . . . . . # . . . #
# # # # # # # # # # #

```

Os símbolos # representam as paredes do labirinto, e os pontos (.) representam quadrados pelo caminho que, possivelmente, levam à saída do labirinto.

Existe um algoritmo simples usado para percorrer o labirinto que garante que encontremos a saída (supondo que exista uma). Se não existir uma saída, você chegará ao ponto de partida novamente. Coloque sua mão direita na parede à sua direita e comece a caminhar para a frente.

Nunca remova sua mão da parede. Se o labirinto virar para a direita, você seguirá a parede para a direita. Desde que não remova sua mão da parede, por fim você chegará à saída do labirinto. Pode ser que haja um caminho mais curto que aquele que você tomou, mas o mais longo garante sua saída do labirinto.

Escreva a função recursiva `mazeTraverse` para percorrer o labirinto. A função deverá receber como argumentos um array de 12 por 12 caracteres que represente o labirinto e o local em que labirinto começa. À medida que `mazeTraverse` tenta localizar a saída do labirinto, ele deverá colocar o caractere X em cada quadrado no caminho. A função deverá exibir o labirinto após cada movimento para que o usuário possa observar enquanto o labirinto é resolvido.

7.23 Geração aleatória de labirintos. Escreva uma função `mazeGenerator` que use um array de 12 por 12 caracteres de duplo subscrito como argumento e que produza um labirinto aleatoriamente. A função também deverá fornecer os locais em que o labirinto começa e termina. Experimente sua função `mazeTraverse` a partir do Exercício 7.22 usando diversos labirintos gerados aleatoriamente.

7.24 Labirintos de qualquer tamanho. Generalize as funções `mazeTraverse` e `mazeGenerator` dos exercícios 7.22 e 7.23 para processar labirintos de qualquer largura e altura.

7.25 Arrays de ponteiros para funções. Reescreva o programa da Figura 6.22 para que seja possível fazer uso de uma interface controlada por menus. O programa deverá oferecer ao usuário as quatro opções a seguir:

Digite uma escolha:

- 0 Imprime o array de notas
- 1 Acha a menor nota
- 2 Acha a maior nota
- 3 Imprime a média de todos os testes para cada aluno
- 4 Encerra o programa

O uso de arrays de ponteiros para funções tem uma restrição: todos os ponteiros precisam ter o mesmo tipo. Os ponteiros precisam servir para funções com o mesmo tipo de retorno, que recebe argumentos do mesmo tipo. Por esse motivo, as funções na Figura 6.22 precisam ser modificadas de modo que cada uma delas retorne o mesmo tipo e use os mesmos parâmetros. Modifique as funções `minimum` e `maximum` para imprimir os valores mínimo e máximo, e não retornar nada. Para a opção 3, modifique a função `average` da Figura 6.22 para mostrar a média de cada aluno (e não de um aluno específico). A função `average` não deverá retornar nada, mas

deverá usar os mesmos parâmetros de `printArray`, `minimum` e `maximum`. Armazene os ponteiros para as quatro funções no array `processGrades` e use a escolha feita pelo usuário como o subscrito do array que chama cada função.

7.26 O que o programa a seguir faz?

```
1  /* ex07_26.c */
2  /* O que esse programa faz? */
3  #include <stdio.h>
4
5  int mystery3( const char *s1, const
    char *s2 ); /* protótipo */
6
7  int main( void )
8  {
9      char string1[ 80 ]; /* cria array
    char */
10     char string2[ 80 ]; /* cria array
    char */
11
```

```
12     printf( "Digite duas strings: " );
13     scanf( "%s%s", string1, string2
    );
14     printf( "O resultado é %d\n", mys-
    tery3( string1, string2 ) );
15     return 0; /* indica conclusão bem-
    sucedida */
16 } /* fim do main */
17
18 int mystery3( const char *s1, const
    char *s2 )
19 {
20     for ( ; *s1 != '\0' && *s2 != '\0';
    s1++, s2++ ) {
21
22         if ( *s1 != *s2 ) {
23             return 0;
24         } /* fim do if */
25     } /* fim do for */
26
27     return 1;
28 } /* fim da função mystery3 */
```

Seção especial de exercícios: criando seu próprio computador

A partir dos próximos problemas, faremos um desvio temporário do mundo da programação em linguagem de alto nível. 'Abriremos a tampa' de um computador e examinaremos sua estrutura interna. Apresentaremos a programação em linguagem de máquina e escreveremos vários programas nessa linguagem. Para tornar essa uma experiência especialmente valiosa, montaremos um computador (por meio da técnica de *simulação* baseada no software) por meio do qual você poderá executar seus programas em linguagem de máquina!

7.27 Programação em linguagem de máquina. Vamos criar um computador a que chamaremos de Simpletron. Como seu nome sugere, ele é uma máquina simples, porém, como veremos em breve, também é poderosa. O Simpletron roda programas escritos na única linguagem que ele entende diretamente — ou seja, a Simpletron Machine Language, ou SML, para abreviar.

O Simpletron contém um *acumulador* — um 'registrador especial' em que as informações são colocadas antes que o Simpletron as utilize em cálculos ou as examine de diversas maneiras. No Simpletron, toda a informação é tratada em termos de *palavras*. Uma palavra é um número decimal de quatro dígitos com sinal, como +3364, -1293, +0007, -0001 e assim por diante. O Simpletron é equipado com uma memória de 100 palavras, e essas palavras são referenciadas por seus números de local 00, 01, ..., 99.

Antes de executar um programa SML, temos que *carregar*, ou colocar, o programa na memória. A primeira instrução (ou comando) de cada programa SML é sempre colocada no local 00.

Cada instrução escrita em SML ocupa uma palavra da memória do Simpletron (e, portanto, as instruções são números decimais de quatro dígitos com sinal). Consideramos que o sinal de uma instrução SML é sempre de adição, porém o sinal de uma palavra de dados pode ser tanto de adição quanto de subtração. Cada local na memória do Simpletron pode conter uma instrução, um valor de dados usado por um programa ou uma área da memória não utilizada (e, portanto, indefinida). Os dois primeiros dígitos de cada instrução SML consistem no *código de operação*, que especifica a operação a ser realizada. Os códigos de operação em SML estão resumidos na Figura 7.32.

Código de operação	Significado
<i>Operações de entrada/saída:</i>	
#define READ 10	Lê uma palavra do terminal para um local específico na memória.
#define WRITE 11	Escreve uma palavra de um local específico na memória para o terminal.

Figura 7.32 ■ Códigos de operação na Simpletron Machine Language (SML). (Parte I de 2.)

Código de operação	Significado
<i>Operações de carregamento/armazenamento:</i>	
#define LOAD 20	Carrega uma palavra de um local específico na memória para o acumulador.
#define STORE 21	Armazena uma palavra do acumulador para um local específico na memória.
<i>Operações aritméticas:</i>	
#define ADD 30	Soma uma palavra de um local específico na memória à palavra no acumulador (deixa o resultado no acumulador).
#define SUBTRACT 31	Subtrai uma palavra de um local específico na memória da palavra no acumulador (deixa o resultado no acumulador).
#define DIVIDE 32	Divide uma palavra de um local específico na memória pela palavra no acumulador (deixa o resultado no acumulador).
#define MULTIPLY 33	Multiplica uma palavra de um local específico na memória pela palavra no acumulador (deixa o resultado no acumulador).
<i>Operações de transferência de controle:</i>	
#define BRANCH 40	Desvia para um local específico na memória.
#define BRANCHNEG 41	Desvia para um local específico na memória se o acumulador for negativo.
#define BRANCHZERO 42	Desvia para um local específico na memória se o acumulador for zero.
#define HALT 43	Para (<i>halt</i>), ou seja, o programa concluiu sua tarefa.

Figura 7.32 ■ Códigos de operação na Simpletron Machine Language (SML). (Parte 2 de 2.)

Os dois últimos dígitos de uma instrução SML consistem no *operando*, que é o endereço do local de memória que contém a palavra à qual a operação se aplica. Agora, consideremos diversos programas SML simples. O programa SML a seguir lê dois números do teclado, calcula e imprime sua soma.

Exemplo 1		
Local	Número	Instrução
00	+1007	(Lê A)
01	+1008	(Lê B)
02	+2007	(Carrega A)
03	+3008	(Soma B)
04	+2109	(Armazena C)
05	+1109	(Escreve C)

06	+4300	(Halt)
07	+0000	(Variável A)
08	+0000	(Variável B)
09	+0000	(Resultado C)

A instrução +1007 lê o primeiro número do teclado e o coloca no local 07 (que foi inicializado em zero). Depois, +1008 lê o próximo número para o local 08. A instrução *load*, +2007, coloca o primeiro número no acumulador, e a instrução *add*, +3008, soma o segundo número ao número no acumulador. *Todas as instruções aritméticas da SML deixam seus resultados no acumulador.* A instrução *store*, +2109, coloca o resultado de volta ao local de memória 09, do qual a instrução *write*, +1109, apanha o número e o imprime (como um número decimal de quatro dígitos com sinal). A instrução *halt*, +4300, encerra a execução.

O programa SML a seguir lê dois números do teclado, determina e imprime o valor maior. Observe o uso da instrução +4107 como uma transferência de controle condicional, da mesma forma que a estrutura *if* em C.

Exemplo 2		
Local	Número	Instrução
00	+1009	(Lê A)
01	+1010	(Lê B)
02	+2009	(Carrega A)
03	+3110	(Subtrai B)
04	+4107	(Desvia, se negativo, para 07)
05	+1109	(Escreve A)
06	+4300	(Halt)
07	+1110	(Escreve B)
08	+4300	(Halt)
09	+0000	(Variável A)
10	+0000	(Variável B)

Agora, escreva programas SML que realizem cada uma das tarefas a seguir:

- Use um loop controlado por sentinela que leia 10 inteiros positivos, e calcule e imprima sua soma.
- Use um loop controlado por contador que leia sete números, alguns positivos e alguns negativos, e calcule e imprima sua média.
- Leia uma série de números e determine e imprima o maior deles. O primeiro número lido indica quantos números deverão ser processados.

7.28 Um simulador de computador: A princípio, pode parecer incrível, mas ao resolver esse problema você estará construindo seu próprio computador. Não, você não terá

que soldar componentes. Na verdade, você usará a técnica poderosa da *simulação baseada no software* para criar um *modelo de software* do Simpletron. Você não ficará desapontado. Seu simulador do Simpletron transformará o computador que você está usando em um Simpletron, e você realmente poderá executar, testar e depurar os programas SML que foram escritos no Exercício 7.27.

Ao executar seu simulador do Simpletron, ele deverá começar imprimindo:

```
*** Bem vindo ao Simpletron! ***
*** Favor digitar seu programa, uma instrução ***
*** (ou palavra de dados) por vez. Mostrarei ***
*** o número do local e uma interrogação (?). ***
*** Você, então, deverá digitar a palavra para esse ***
*** local. Digite a sentinela -99999 para ***
*** encerrar a entrada do seu programa. ***
```

Simule a memória do Simpletron com um array de subscrito único, *memory*, que contenha 100 elementos. Agora, suponha que o simulador esteja sendo executado. Examinemos o diálogo enquanto digitamos o programa do Exemplo 2 do Exercício 7.27:

```
00 ? +1009
01 ? +1010
02 ? +2009
03 ? +3110
04 ? +4107
05 ? +1109
06 ? +4300
07 ? +1110
08 ? +4300
09 ? +0000
10 ? +0000
11 ? -99999
*** Carga do programa concluída ***
*** Iniciando execução do programa ***
```

Agora, o programa SML foi colocado (ou carregado) no array *memory*. O Simpletron executa o programa SML. Ele começa com a instrução no local 00 e continua a sequência, a menos que seja direcionado para alguma outra parte do programa por uma transferência de controle.

Use a variável *accumulator* para representar o registrador do acumulador. Use a variável *instructionCounter* para registrar o local na memória que contém a instrução que está sendo executada. Use a variável *operationCode* para indicar a operação que está sendo executada no momento — ou seja, os dois dígitos da esquerda da palavra de instrução. Use a variável *operand* para indicar o local da memória em que a instrução está operando no momento. Assim, *operand* são os dois dí-

gitos mais à direita da instrução que está sendo executada no momento. Não execute instruções diretamente da memória. Em vez disso, transfira a próxima instrução a ser executada da memória para uma variável chamada *Register*. Depois, ‘recolha’ os dois dígitos da esquerda e coloque-os na variável *operationCode*, e ‘recolha’ os dois dígitos da direita e coloque-os em *operand*.

Quando o Simpletron iniciar sua execução, os registradores especiais serão inicializados da seguinte forma:

<i>accumulator</i>	+0000
<i>instructionCounter</i>	00
<i>instructionRegister</i>	+0000
<i>operationCode</i>	00
<i>Operand</i>	00

Agora, vamos ‘caminhar’ pela execução da primeira instrução SML, +1009, no local de memória 00. Isso é chamado *ciclo de execução da instrução*.

O *instructionCounter* nos diz o local da próxima instrução a ser executada. *Buscamos* o conteúdo desse local a partir de *memory* usando a instrução C

```
instructionRegister = memory[ instructionCounter ] ;
```

O código da operação e o operando são extraídos do registrador de instrução por meio das instruções

```
operationCode = instructionRegister / 100;
operand = instructionRegister % 100;
```

Agora, o Simpletron precisa determinar se o código da operação é realmente uma *leitura* (ou uma *escrita*, uma *carga* e assim por diante). Um *switch* diferencia entre as doze operações da SML.

Na estrutura *switch*, o comportamento de diversas instruções SML é simulado da seguinte forma (deixamos as outras para o leitor):

```
ler: scanf( "%d", &memory[ operand ] );
carregar: accumulator = memory[ operand ];
somar: accumulator += memory[ operand ];
diversas instruções de desvio: Discutiremos em breve.
balt: Essa instrução imprime a mensagem
```

```
*** Execução do Simpletron encerrada ***
```

e depois imprime o nome e o conteúdo de cada registrador, bem como o conteúdo completo da memória. Essa listagem normalmente é chamada de *dump do computador*. Para ajudá-lo a programar sua função de *dump*, uma amostra do formato de *dump* aparece na Figura 7.33. Um *dump*, após a execução do programa Simpletron, mostraria os valores reais das instruções e os valores dos dados no momento em que a execução foi encerrada.

REGISTERS:

```

accumulator      +0000
instructionCounter    00
instructionRegister  +0000
operationCode       00
operand            00

```

MEMORY:

	0	1	2	3	4	5	6	7	8	9
0	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000
10	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000
20	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000
30	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000
40	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000
50	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000
60	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000
70	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000
80	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000
90	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000	+0000

Figura 7.33 ■ Exemplo do formato de dump do Simpletron.

Prosseguiremos com a execução da primeira instrução do nosso programa, a saber, o +1009 no local 00. Conforme indicamos, a estrutura `switch` simula isso ao executar a instrução C

```
scanf( "%d", &memory[ operand ] );
```

O ponto de interrogação (?) deve ser exibido na tela antes que o `scanf` seja executado, como modo de pedir a entrada do usuário. O Simpletron espera que o usuário digite um valor e depois pressione a tecla *Enter*. O valor é, então, lido no local 09.

Nesse ponto, a simulação da primeira instrução foi concluída. Tudo o que resta é preparar o Simpletron para a execução da próxima instrução. Como a instrução recém-executada não foi uma transferência de controle, precisamos simplesmente incrementar o registrador do contador de instrução da seguinte forma:

```
++instructionCounter;
```

Isso conclui a execução simulada da primeira instrução. O processo inteiro (ou seja, o ciclo de execução da instrução) começa novamente com a busca da próxima instrução a ser executada.

Agora, consideremos como as instruções de desvio — as transferências de controle — são simuladas. Tudo o que precisamos é ajustar o valor no contador de instrução de

forma adequada. Portanto, a instrução de desvio incondicional (40) é simulada dentro do `switch` como

```
instructionCounter = operand;
```

A instrução condicional de ‘desvio se acumulador é zero’ é simulada como

```
if ( accumulator == 0 ) {
    instructionCounter = operand;
}
```

Nesse ponto, você deverá implementar seu simulador Simpletron e executar os programas SML que escreveu no Exercício 7.27. Você poderá embelezar a SML com outros recursos e oferecê-los em seu simulador.

Seu simulador deverá verificar os diversos tipos de erros. Durante a fase de carga do programa, por exemplo, cada número que o usuário digita na `memory` do Simpletron deverá estar no intervalo -9999 a +9999. Seu simulador deverá usar um `while` para testar se cada número inserido está nessa faixa e, se não estiver, deverá continuar pedindo ao usuário que informe o número novamente até que um número correto seja digitado.

Durante a fase de execução, seu simulador deverá verificar vários erros sérios, como tentativas de divisão por zero, tentativas de execução de códigos de operação inválidos.

lidos e estouros do acumulador (ou seja, operações aritméticas que resultam em valores maiores que +9999 ou menores que -9999). Esses erros sérios são chamados *erros fatais*. Quando um erro fatal é detectado, seu simulador deverá imprimir uma mensagem de erro como:

```
*** Tentativa de divisão por zero ***
```

```
*** Execução do Simpletron encerrada de forma anormal. ***
```

e deverá imprimir um dump completo do computador no formato que apresentamos anteriormente. Isso ajudará o usuário a localizar o erro no programa.

7.29 Modificações no simulador do Simpletron. No Exercício 7.28, você escreveu uma simulação de software de um computador que executa programas escritos na Simpletron Machine Language (SML). Nesse exercício, propomos várias modificações e melhorias no Simulador do Simpletron. Nos exercícios 12.26 e 12.27, propomos criar um compilador que converte programas escritos em uma linguagem de programação de alto nível (uma variação do BASIC) para a linguagem de máquina do Simpletron. Algumas das modificações e melhorias a seguir podem ser necessárias para executar os programas produzidos pelo compilador.

- a) Estenda a memória do Simulador do Simpletron para que ele contenha 1000 locais de memória, permitindo que ele trate de programas maiores.
- b) Permita que o simulador realize cálculos de módulo. Isso exige uma instrução adicional na linguagem de máquina do Simpletron.
- c) Permita que o simulador realize cálculos de exponenciação. Isso exige uma instrução adicional na linguagem de máquina do Simpletron.
- d) Modifique o simulador para que seja possível usar valores hexadecimais em vez de valores inteiros para

representar as instruções na linguagem de máquina do Simpletron.

- e) Modifique o simulador para permitir a saída de uma nova linha. Isso exige uma instrução adicional na linguagem de máquina do Simpletron.
- f) Modifique o simulador para que ele possa processar valores de ponto flutuante, além de valores inteiros.
- g) Modifique o simulador fazendo com que seja possível lidar com a input de string. [Dica: cada palavra do Simpletron pode ser dividida em dois grupos, cada um mantendo um inteiro de dois dígitos. Cada inteiro de dois dígitos representa o equivalente decimal ASCII de um caractere. Inclua uma instrução em linguagem de máquina que entrará com uma string, e armazenará essa string, a partir de um local específico da memória do Simpletron. A primeira metade da palavra nesse local será um contador do número de caracteres na string (ou seja, o comprimento da string). Cada meia palavra seguinte conterá um caractere ASCII expresso como dois dígitos decimais. A instrução em linguagem de máquina converterá cada caractere em seu equivalente ASCII e o atribuirá a uma meia palavra.]
- h) Modifique o simulador para que seja possível lidar com a saída de strings armazenadas no formato da parte (g). [Dica: inclua uma instrução em linguagem de máquina que imprima uma string que começa em um local específico da memória do Simpletron. A primeira metade da palavra nesse local é o comprimento da string em caracteres. Cada meia palavra seguinte contém um caractere ASCII expresso como dois dígitos decimais. A instrução em linguagem de máquina verifica o comprimento e imprime a string, traduzindo cada número de dois dígitos em seu caractere equivalente.]

Exercícios de array de ponteiro para função

7.30 Cálculo da circunferência do círculo, da área do círculo e do volume da esfera usando ponteiros para função. Usando as técnicas que você aprendeu na Figura 7.28, crie um programa baseado em texto, controlado por menu, que permita que o usuário escolha se irá calcular a circunferência de um círculo, a área de um círculo ou o volume de uma esfera. O programa deverá, então, solicitar do usuário um raio, realizar o cálculo apropriado e exibir o resultado. Use um array de ponteiros para função em que cada ponteiro represente uma função que retorne `void` e receba um parâmetro `double`. Cada uma das funções correspondentes deve exibir mensagens indicando qual cálculo foi realizado, o valor do raio e o resultado do cálculo.

7.31 Execução de cálculos usando ponteiros para função. Usando as técnicas que você aprendeu na Figura 7.28, crie um programa baseado em texto, controlado por menu, que permita que o usuário escolha se vai somar, subtrair, multiplicar ou dividir dois números. O programa deverá, então, solicitar do usuário dois valores `double`, realizar o cálculo apropriado e exibir o resultado. Use um array de ponteiros para função em que cada ponteiro represente uma função, retorne `void` e receba dois parâmetros `double`. Cada uma das funções correspondentes deve exibir mensagens que indiquem o cálculo que foi realizado, os valores dos parâmetros e o resultado do cálculo.

Fazendo a diferença

7.32 Votação. A Internet e a Web vêm permitindo que as pessoas, cada vez mais, estabeleçam redes de contatos, juntem-se a uma causa, expressem opiniões e assim por diante. Em 2008, os candidatos à presidência dos Estados Unidos usaram amplamente a Internet para passar suas mensagens e levantar fundos para suas campanhas. Nesse exercício, você escreverá um programa de votação simples, que permite que os usuários avaliem, de 1 (menos importante) a 10 (mais importante), cinco questões de importância social. Escolha cinco causas que sejam importantes para você (por exemplo, questões políticas, questões de meio ambiente). Use um array unidimensional `topics` (do tipo `char *`) para armazenar as cinco causas. Para resumir as respostas do estudo, use um array bidimensional de 5 linhas e 10 colunas, `responses` (do tipo `int`), com cada linha correspondendo a um elemento no array `topics`. Ao ser executado, o programa deve pedir ao usuário que dê uma nota para cada questão. Peça a seus amigos e familiares que respondam à pesquisa. Depois, faça com que o programa apresente um resumo dos resultados, que inclua:

- a) Um relatório tabular com os cinco tópicos no lado esquerdo e as 10 classificações no topo, listando em cada coluna o número de classificações recebidas para cada tópico.
- b) À direita de cada linha, mostre a média das classificações para essa questão.
- c) Qual questão recebeu a maior pontuação total? Mostre a questão e o total de pontos.

- d) Qual questão recebeu a menor pontuação total? Mostre a questão e o total de pontos.

7.33 Medição de emissão de dióxido de carbono: arrays de ponteiros para função. Usando arrays de ponteiros para função, como você aprendeu neste capítulo, é possível especificar um conjunto de funções que são chamadas com os mesmos tipos de argumentos e que retornam o mesmo tipo de dado. Governos e empresas do mundo inteiro estão se preocupando cada vez mais com as medições de carbono (emissões anuais de dióxido de carbono na atmosfera) de prédios que queimam vários tipos de combustíveis para gerar calor, veículos que queimam combustíveis para gerar potência e outras questões similares. Muitos cientistas culpam esses gases de efeito estufa pelo fenômeno chamado aquecimento global. Crie três funções que ajudem a calcular a medição de emissão de dióxido de carbono de um prédio, um carro e uma bicicleta, respectivamente. Cada função deverá solicitar dados apropriados do usuário e, depois, calcular e exibir as respectivas medições. (Pesquise alguns sites que explicam como calcular as medições de carbono.) As funções não deverão receber parâmetros, mas deverão retornar `void`. Escreva um programa que induza o usuário a digitar o tipo de medição de emissão de dióxido de carbono a ser calculada, e depois chame a função correspondente no array dos ponteiros para função. Para cada tipo de medição, apresente alguma informação de identificação e o objeto que deixa aquela emissão.